A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs

Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento

汪逸阳，2022.3.20

目录

- 介绍
- 算法的准备工作
- 伪代码
- 候选集的计算
- 同构性的检测
- 采用的数据结构
- 实验结果
- 结论

介绍

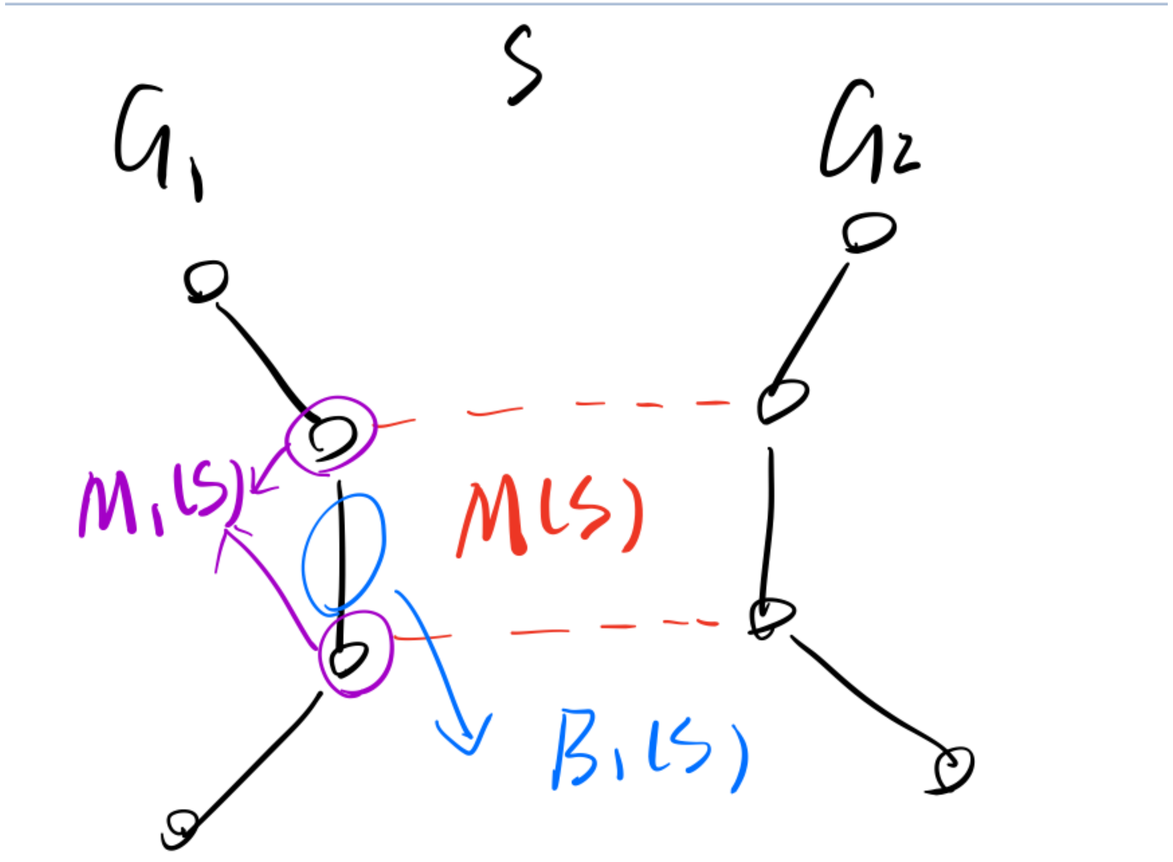
- 如今的诸多科学领域如模式分析、模式认知和计算机视觉对图的应用都不断增多，而从模式分析和认知的角度来看，匹配图或子图从而进行对比是最重要的图处理问题。
- 图匹配可以分为精确匹配和非精确匹配两类，该论文的算法是精确匹配算法。
- 该论文提供了一个决定性算法，可以用来验证图同构和子图同构，该算法具有普适性，因为其对输入的图没有限制。

VF2算法

准备工作

- 图$G_1 = (N_1, B_1), G_2 = (N_2, B_2)$，以及映射集合$M \subset N_1 \times N_2$。我们的目标就是确定并寻找$G_1$（的子图）到$G_2$的同构映射。

- 寻找映射函数的过程可以用状态空间表示(State Space Representation, SSR)来进行描述。每一个状态s确定了一个映射$M(s) \subseteq M$，该映射确定了子图$G_1(s) \subseteq G_1$、$G_2(s) \subseteq G_2$以及其对应的点集$M_1(s), M_2(s)$与边集$B_1(s), B_2(s)$。从一个状态s向后继状态s'的转移即为向s中添加一个新的满足同构关系的点对。



算法描述

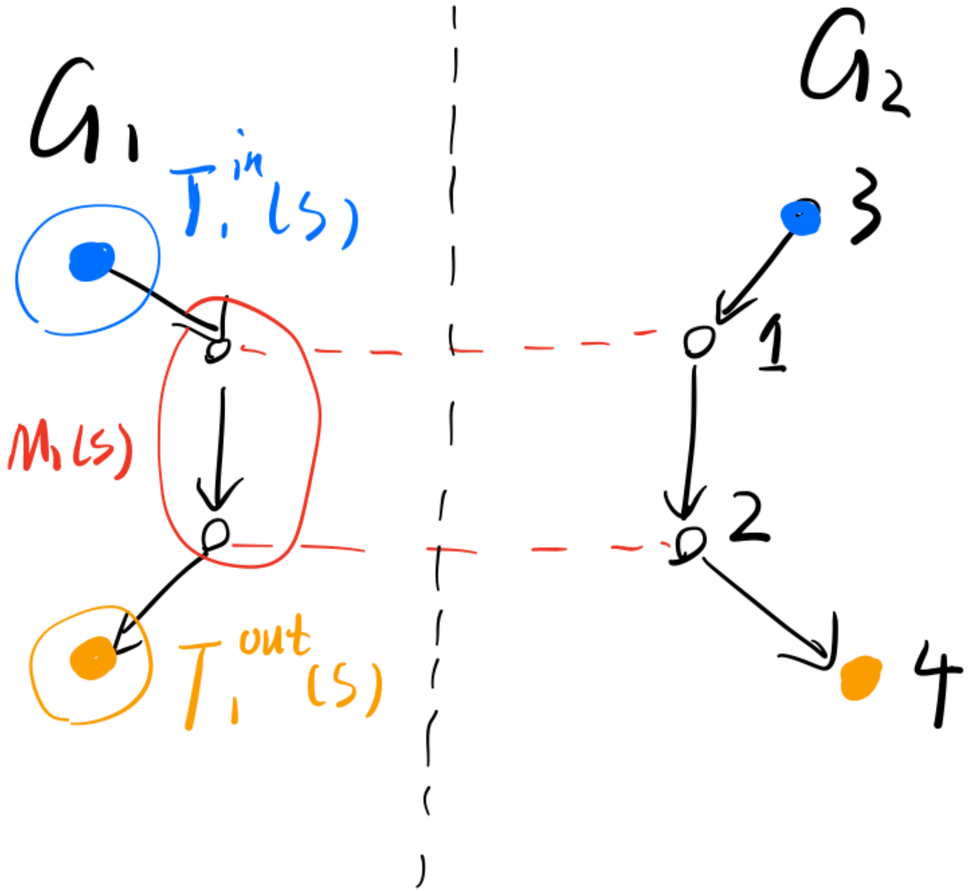
```
PROCEDURE Match(s)
    INPUT:  an intermediate state s; the initial state s0 has M(s0)=∅
    OUTPUT: the mappings between the two graphs

    IF M(s) covers all the nodes of G2 THEN
      OUTPUT M(s)
    ELSE
      Compute the set P(s) of the pairs candidate for inclusion in M(s)
      FOREACH p in P(s)
        IF the feasibility rules succeed for the inclusion of p in M(s) THEN
          Compute the state s´ obtained by adding p to M(s)
          CALL Match(s')
        END IF
      END FOREACH
      Restore data structures
    END IF
END PROCEDURE Match
```

G_2已经和G_1/G_1的子图完全匹配

细节实现

候选集$P(s)$的计算

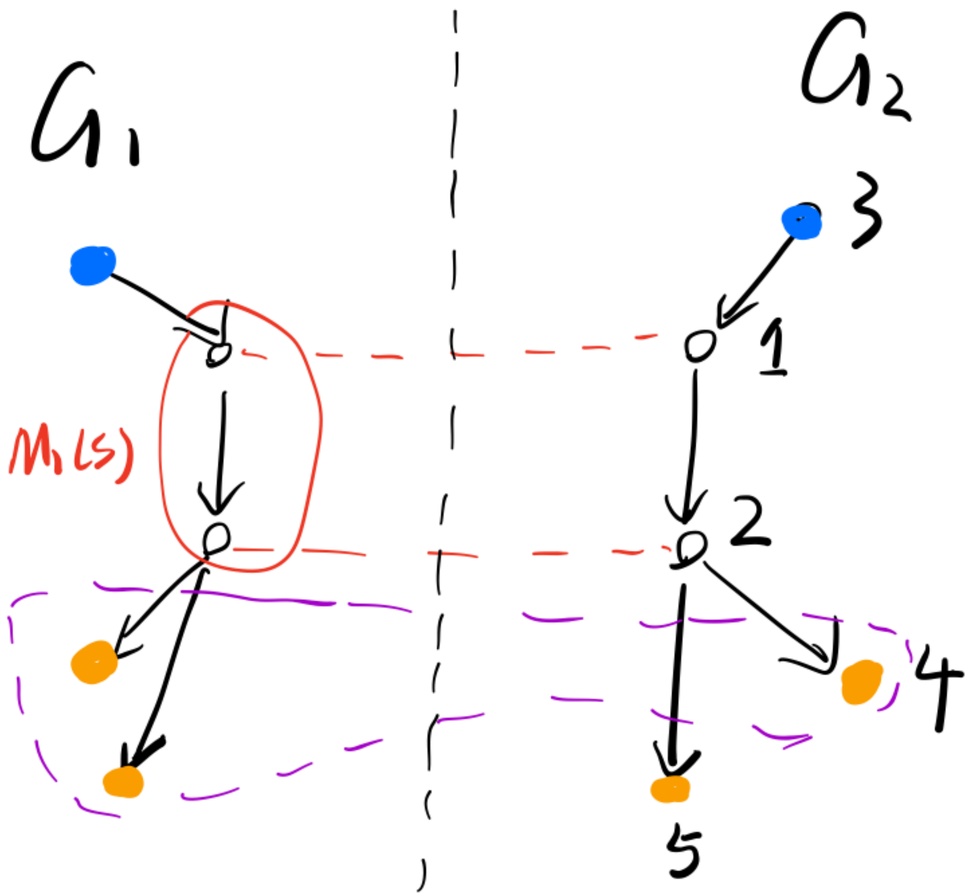
- $T_1^{out}(s)(T_2^{out}(s))$为既为$M_1(s)(M_2(s))$中的点的出边结点又尚不在$M(s)$中的点的集合，同理可定义入边点集$T_1^{in}(s), T_2^{in}(s)$。
- 为了防止在深度周游中重复计算同一个状态s，且构建结果和构建顺序无关，可以在N_2中构建一个全序关系$< N_2, \prec >$。
- 右图中红色代表已经构建好的$M(s)$关系



细节实现

候选集$P(s)$的计算

- (1)当$T_1^{out}(s), T_2^{out}(s)$非空时，$P(s) = T_1^{out} \times \{minT_2^{out}(s)\}$。
- 图中紫色区域即为下一步计算的两种可能映射关系。

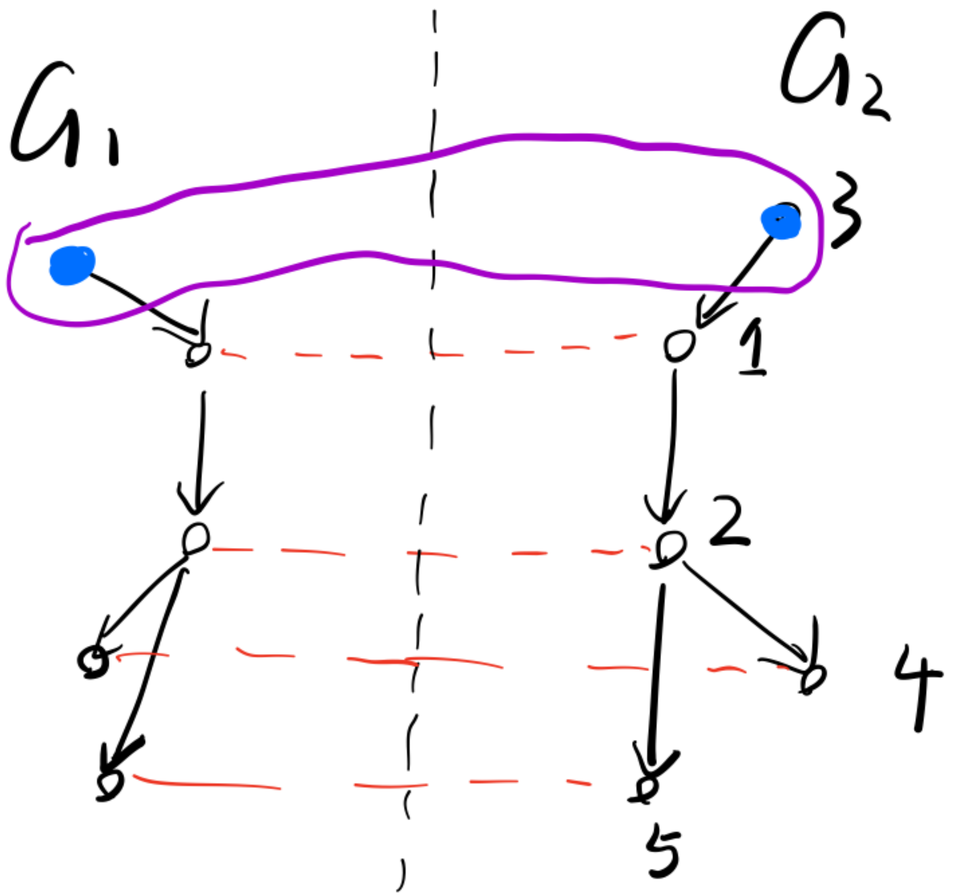


细节实现

候选集$P(s)$的计算

- (2)当$T_1^{in}(s), T_2^{in}(s)$非空时，
  $P(s) = T_1^{in} \times \{minT_2^{in}(s)\}$。

- 图中紫色区域即为下一步计算的两种可能映射关系。

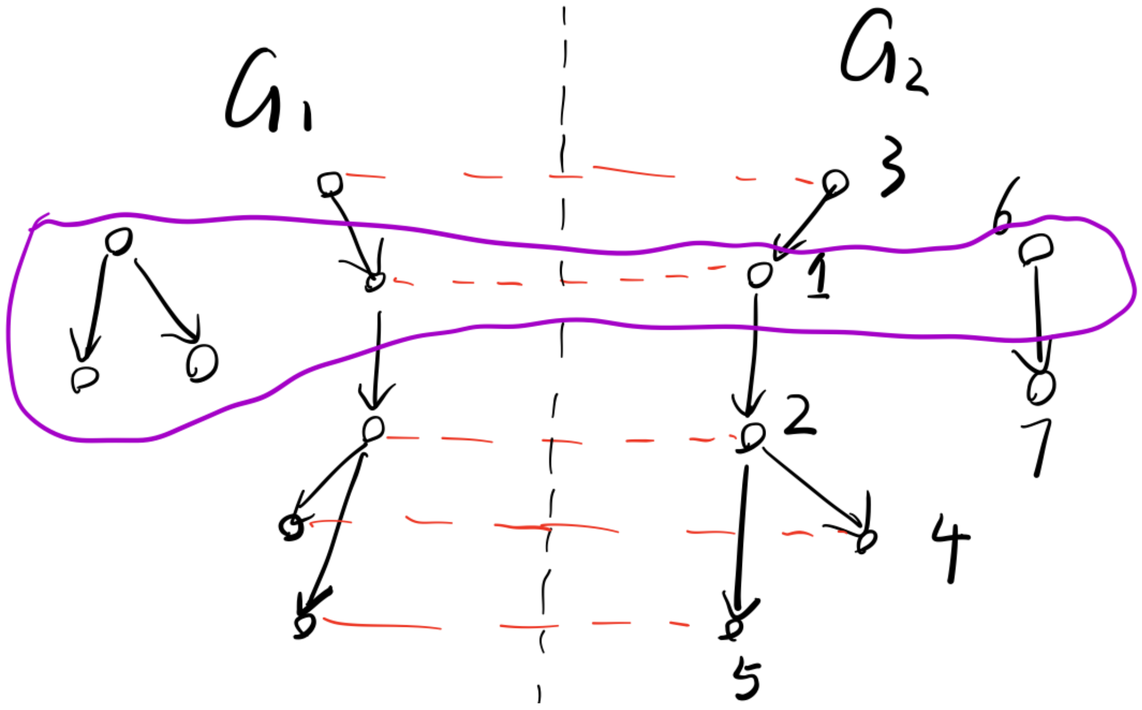


细节实现

候选集$P(s)$的计算

- (3)当四个集合均为空的时候，
  $P(s) = (N_1 - M_1(s)) \times \{min(N_2 - M_2(s))\}$

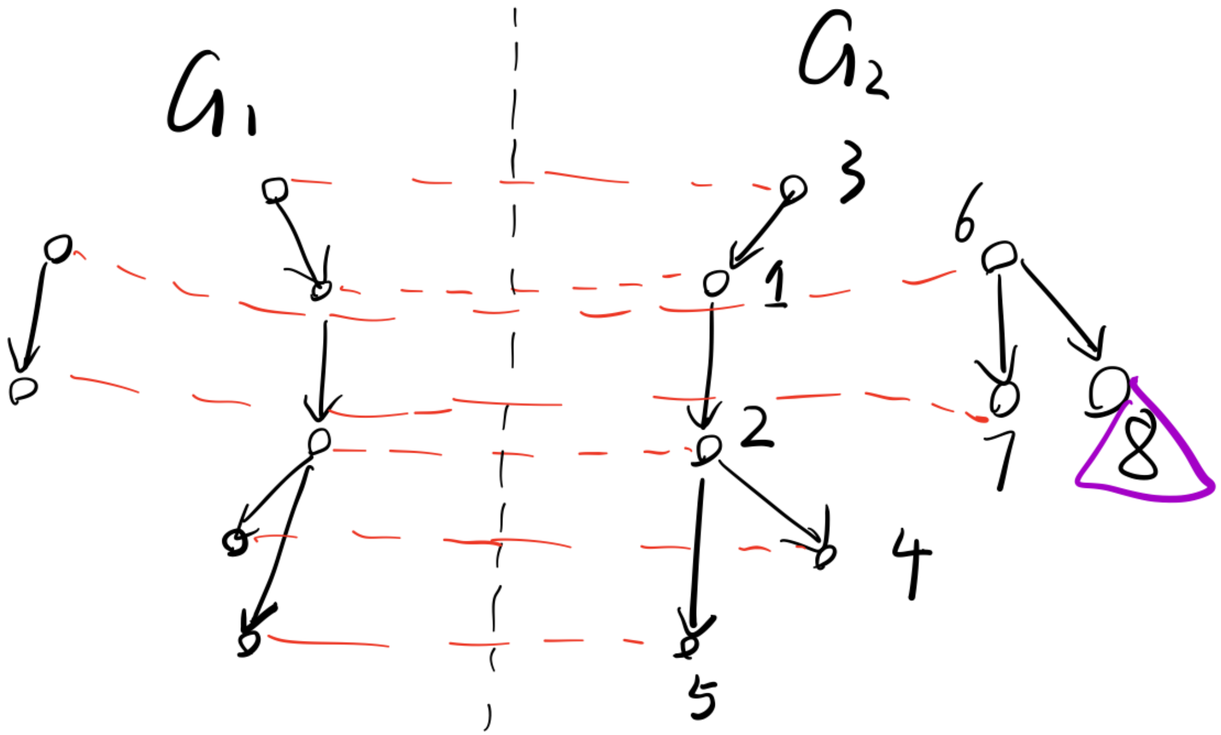
- 图中紫色区域即为下一步计算的两种可能映射关系。



细节实现

候选集$P(s)$的计算

- (4)其他情况则无法保持边结构相同，无法从s继续构造同构。



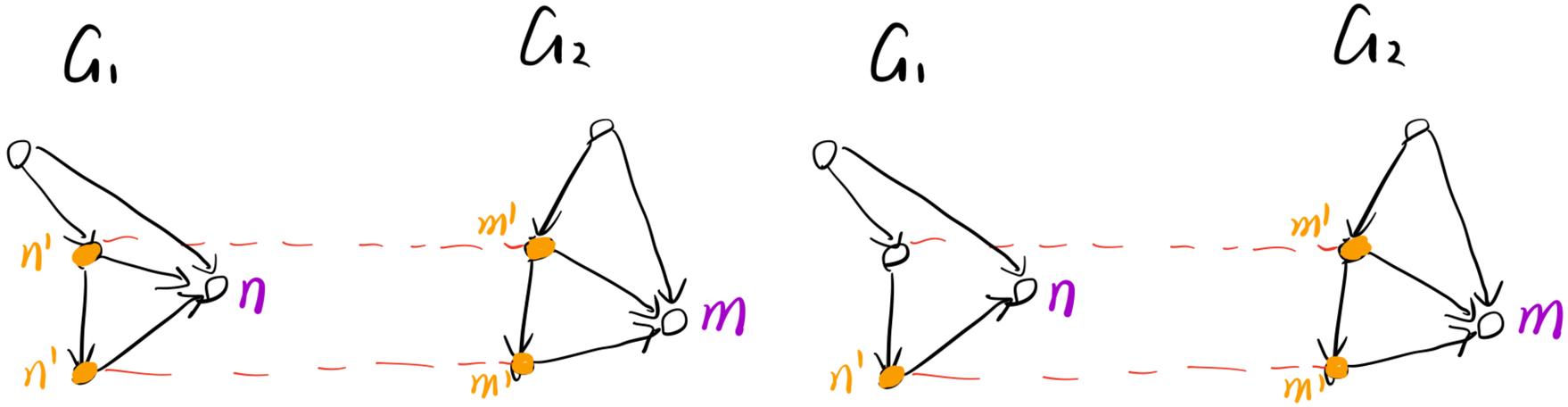
结构可行性的检验(F_{syn}=$R_{pred} \wedge R_{succ} \wedge R_{in} \wedge R_{out} \wedge R_{new}$)

是否保持同构性质的测试

- 检验是否保持了同构性质的表达式：

$$R_{pred}(s,n,m) \Leftrightarrow (\forall n' \in M_1(s) \cap Pred(G_1,n) \exists m' \in Pred(G_2,m) \mid (n',m') \in M(s)) \wedge (\forall m' \in M_2(s) \cap Pred(G_2,m) \exists n' \in Pred(G_1,n) \mid (n',m') \in M(s))$$

- 这个表达式检验新引入的点对(n,m)在$M(s)$中的所有前驱是否对应，将上式的$pred$替换成$succ$即可检验后继是否对应。



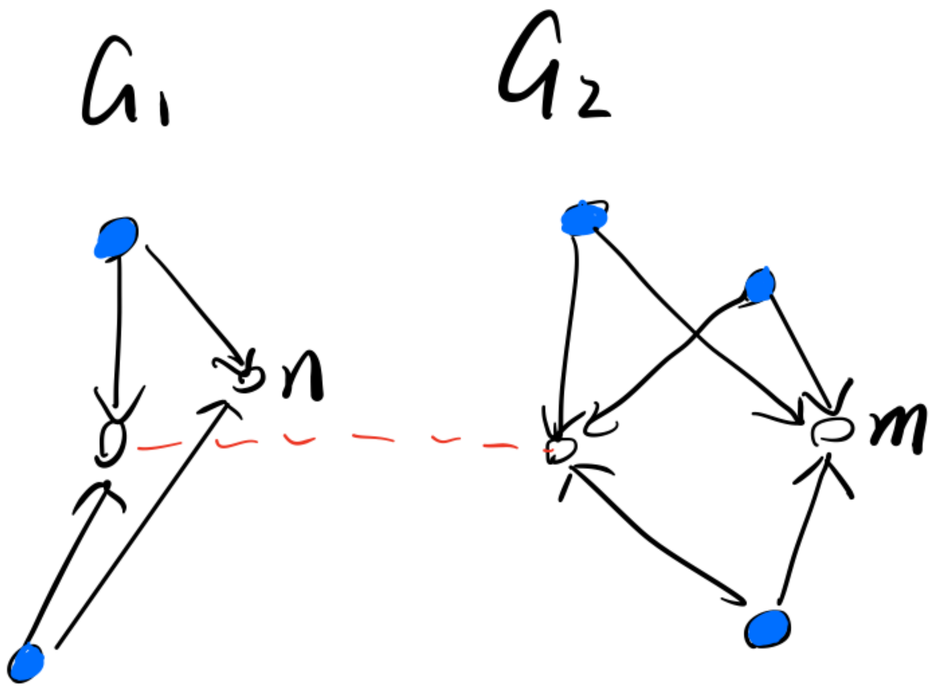
检验成功　　检验失败

结构可行性的检验(F_{syn}=$R_{pred} \wedge R_{succ} \wedge R_{in} \wedge R_{out} \wedge R_{new}$)

用于剪枝的测试1

- $R_{in}(s,n,m) \Leftrightarrow (Card(Succ(G_1,n) \cap T_1^{in}(s) \geq Card(Succ(G_2,m) \cap T_2^{in}(s))) \wedge (Card(Pred(G_1,n) \cap T_1^{in}(s) \geq Card(Pred(G_2,m) \cap T_2^{in}(s)))$

- 这个表达式的含义为，既是新引入的点n的前驱（后继）、又是$G_1(s)$的入（出）邻居的点的个数不能小于既是点m的前驱（后继）、又是$G_2(s)$的入（出）邻居的点的个数，同理将上式的in替换成out也可进行类似的测试。



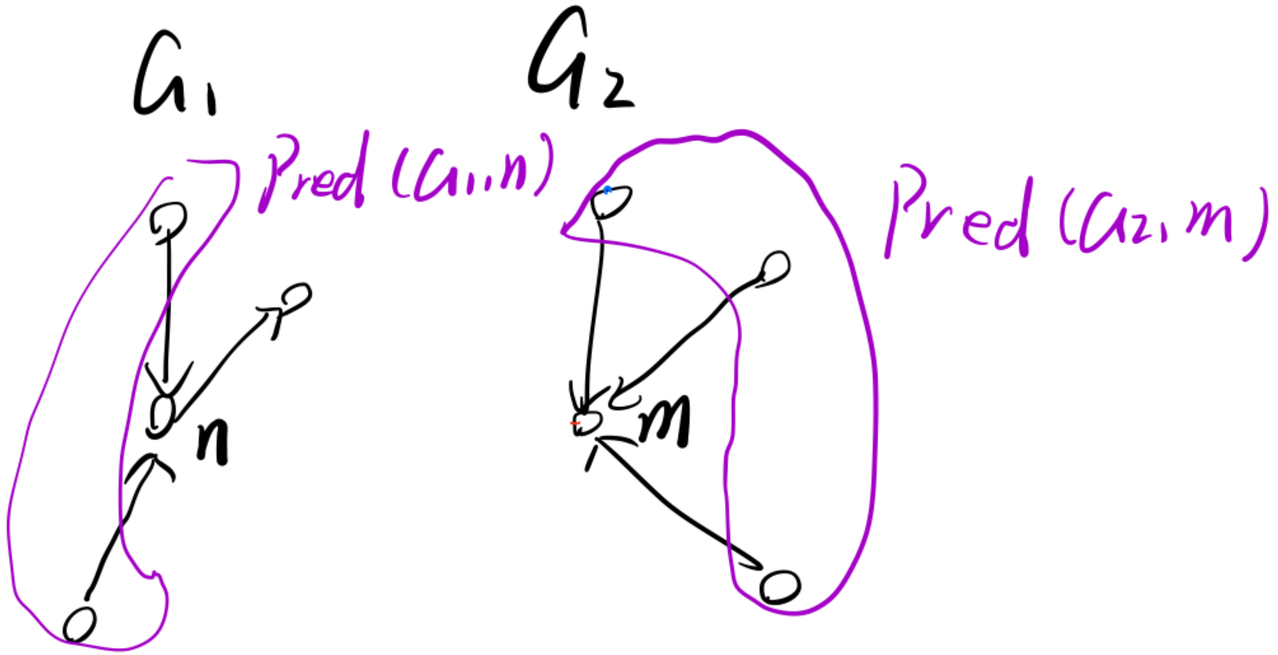
结构可行性的检验(F_{syn}=$R_{pred} \wedge R_{succ} \wedge R_{in} \wedge R_{out} \wedge R_{new}$)

用于剪枝的测试2

- $R_{new}(s,n,m) \Leftrightarrow Card(\tilde{N}_1(s) \cap Pred(G_1,n)) \geq Card(\tilde{N}_2(s) \cap Pred(G_2,m)) \wedge Card(\tilde{N}_1(s) \cap Succ(G_1,n)) \geq Card(\tilde{N}_2(s) \cap Succ(G_2,m))$ （文中的表达式应该是出现了一些纰漏）

- 其中 $\tilde{N}_1(s) = N_1 - M_1(s) - T_1(s)$ 表示目前在G_1中和$M(s)$无直接关系的点，这些点中为新引入的点n的前驱（后继）的点的个数必须不小于与m有同种关系的点，否则也会出现无法（子图）同构的搜索结果。

- 如果将问题缩小为之允许两图同构，那么上述三个剪枝条件的≥需要替换为=。

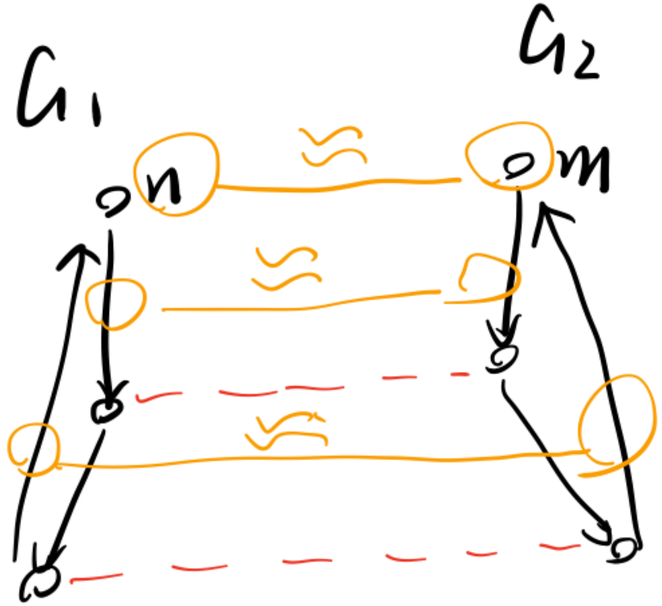


属性可行性的检验

- 对可比的属性值（可以是符号变量、数字变量等），我们用符号≈表示两个属性值可以匹配。定义：

$$F_{sem} \Leftrightarrow n \approx m \wedge \forall (n', m') \in M(s), (n, n') \in B_1 \Rightarrow (n, n') \approx (m, m') \wedge \forall (n', m') \in M(s), (n', n) \in B_1 \Rightarrow (n', n)$$

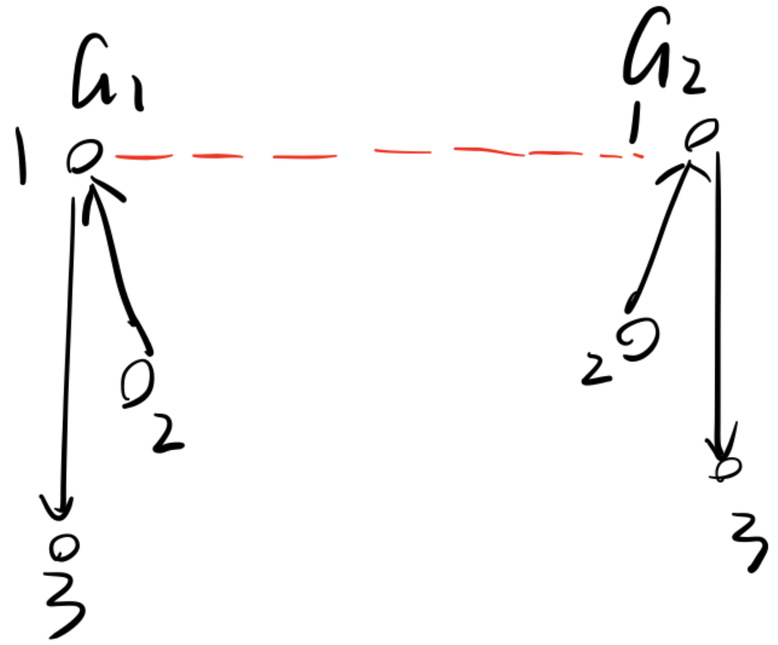
- 还可以引入代价函数$cost(s)$用于计算每个状态的代价，假设新增加的代价都是非负的，我们就可以利用这一信息进行剪枝，也即若新的状态产生的代价比目前已经搜索过的新的状态所产生的最小代价要大的话就不在该状态下继续搜索下去。



数据结构

- 两个向量$core_1, core_2$，维数分别为$|G_1|, |G_2|$，当点n在$M_1(s)$中时，$core_1[n]$中记录了其在G_2中对应的点的索引，否则为$NULL_NODE$。

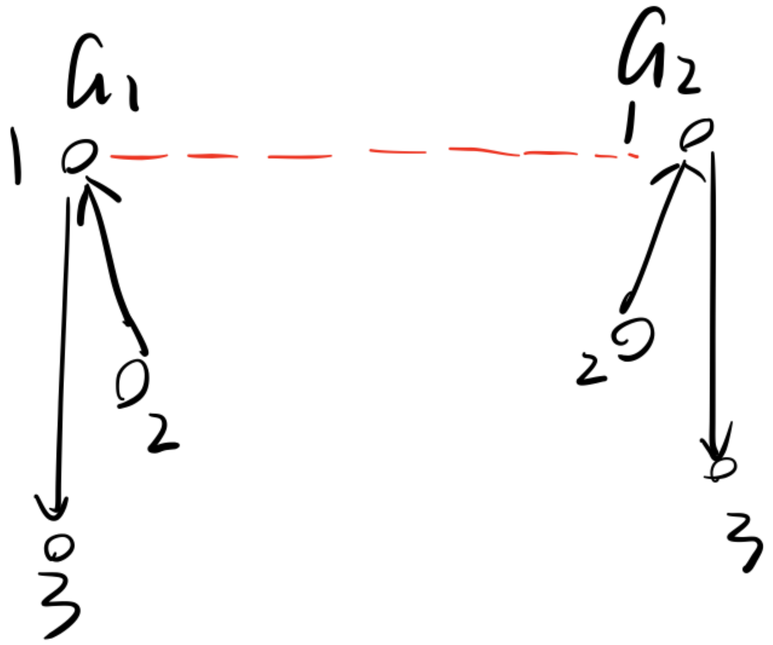
- 四个向量in_1, in_2, out_1, out_2。当点n在$M_1(s)$或$T_1^{in}(s)$中时，$in_1[n]$为非零量，同理可对其他向量进行类似的定义。（为什么不让in_1和$T_1^{in}(s)$等价：为了保持后文所说的非零不变性）



	1	2	3
core 1	1	\	\
core 2	1	\	\
in 1	1	1	0
in 2	1	1	0
out 1	1	0	1
out 2	1	0	1

数据结构

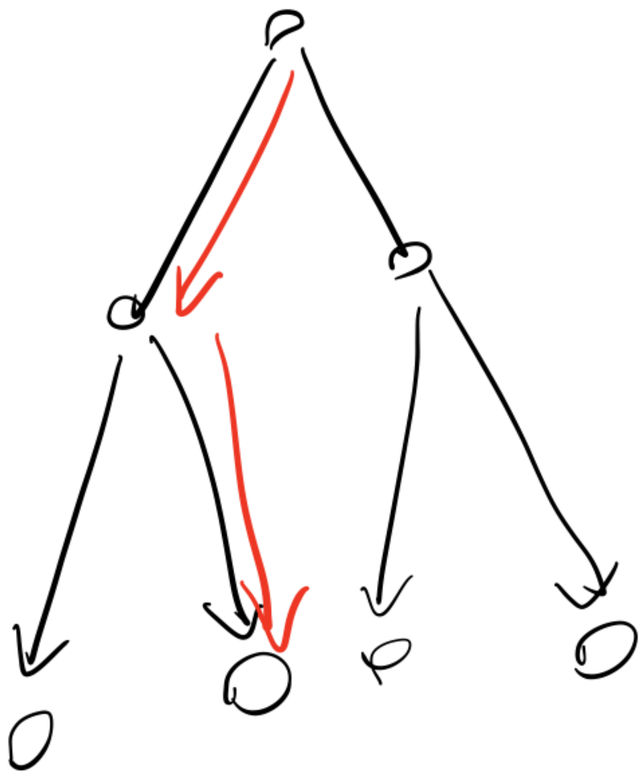
- 通过该数据结构，对一个结点n在是否属于上文中提到的集合中的判定就可以在常数时间内完成，因此$P(s)$的计算可以在$O(|N_1|+|N_2|)$的时间完成。而$F(s,n,m)$的判定时间也为一个与新引入的点n相邻的点的个数成比例的时间（找到前驱/后继，判断其的集合从属关系，再判断其在映射中是否有对应的点）。



	1	2	3
core 1	1	\	\
core 2	1	\	\
in 1	1	1	0
in 2	1	1	0
out 1	1	0	1
out 2	1	0	1

数据结构

- 注意到对一个状态s以及上述的几个向量，若一个向量的元素是非空（0）的，那么其在s状态所确定的搜索树的子树的结点的状态均保持非空，结合DFS策略和回溯法即可避免为每一个状态额外分配新的空间来储存其对应的向量，其在整个算法中占用的空间为$\Theta(n)$；注意到搜索树的深度不会超过结点总数N，故DFS策略保证了在函数栈中保存的状态数最多为N，而**每个状态只需要保存常数空间的信息**，因此整个算法的空间复杂度为$\Theta(n)$



数据结构

- 注意到对一个状态s以及上述的几个向量，若一个向量的元素是非空（0）的，那么其在s状态所确定的搜索树的子树的结点的状态均保持非空
避免
间和
算法

TABLE 1
Spatial and Time Complexity of VF2 and of Ullmann's Algorithm in the Best and Worst Case

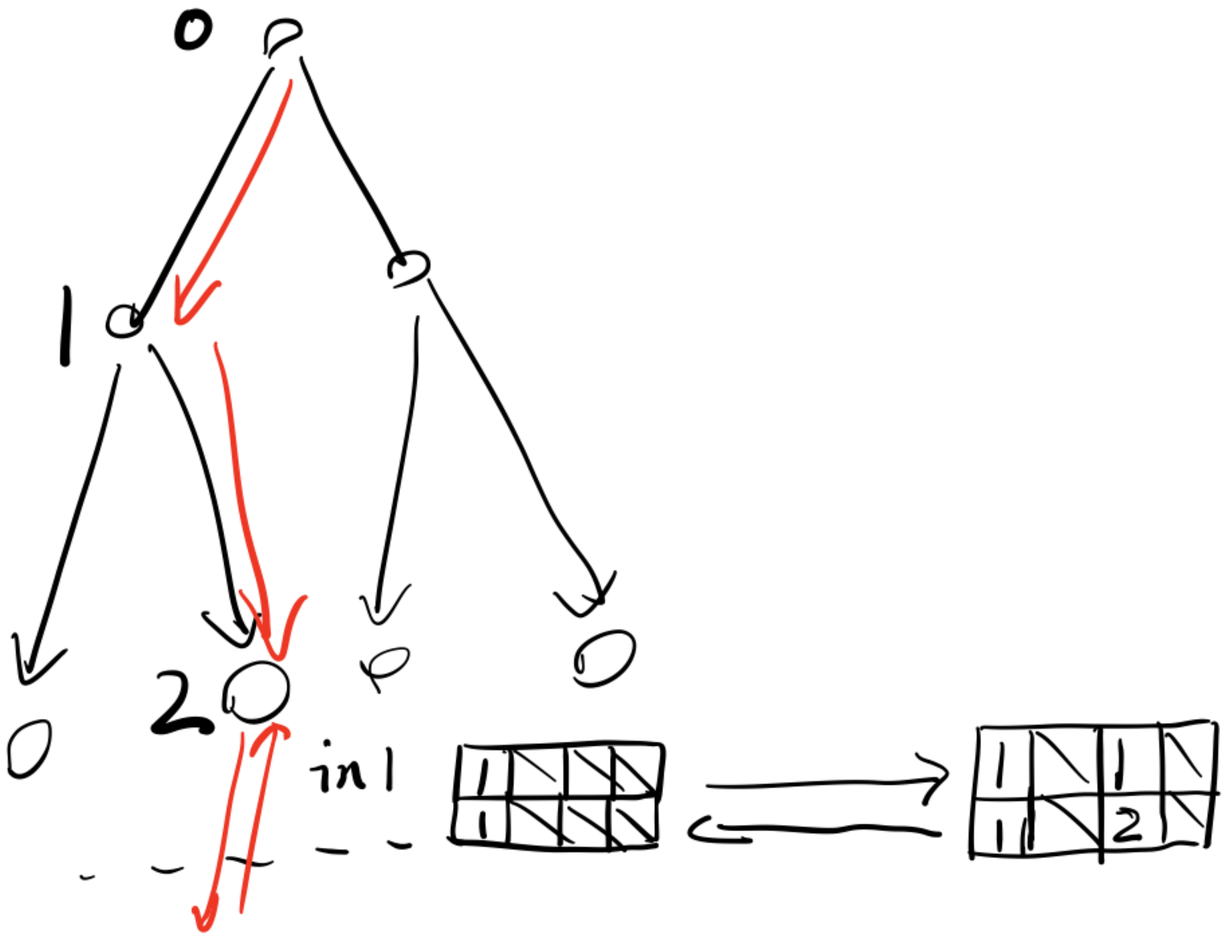
	VF2		Ullmann's algorithm	
Complexity	**Best Case**	**Worst Case**	**Best Case**	**Worst Case**
Time	$\Theta(N^2)$	$\Theta(N!\,N)$	$\Theta(N^3)$	$\Theta(N!\,N^2)$
Spatial	$\Theta(N)$	$\Theta(N)$	$\Theta(N^3)$	$\Theta(N^3)$

搜索树的深度不会超过结点总数N，故DFS策略保证了在函数栈中保存的状态数最多为N，而**每个状态只需要保存常数空间的信息**，因此整个算法的空间复杂度为$\Theta(n)$

关于如何在常数空间做到回溯

- 将向量发生更改的DFS深度记录下来（可以记录在原来输入的图的点处，或者为原先的每个向量的每个元素多增加一个深度记录位，这样子额外的空间依旧是$\Theta(N)$的），在回溯的时候可以通过比对DFS的深度来恢复该阶段的向量值，这样子需要遍历原先新增结点n的所有前驱和后继，最坏情况下的时间复杂度为$O(N)$。

- 如果图的正则性较强，那么一次回溯的时间在平均意义下能控制在常数时间内完成。（从后面的实验结果中也可以看出VF2算法对高正则性的图的匹配具有优势）



实验结果

图同构

TABLE 2
A Comparison between VF2 and Nauty as a Function of the Graph Size and of the Kind of the Graphs

	Randomly Connected			2D (regular and irregular) Mesh				Bounded valence		
Nodes	η=0.01	η=0.05	η=0.1	regular	ρ=0.2	ρ=0.4	ρ=0.6	ν=3	ν=6	ν=9
20	VF2	VF2	Nauty	VF2	Nauty	Nauty	Nauty	Nauty	Nauty	Nauty
40	VF2	Nauty	Nauty	VF2	VF2	Nauty	Nauty	Nauty	Nauty	Nauty
60	VF2	Nauty	Nauty	VF2	VF2	VF2	Nauty	VF2	Nauty	Nauty
80	VF2	Nauty	Nauty	VF2	VF2	VF2	VF2	VF2	Nauty	Nauty
100	VF2	Nauty	Nauty	VF2	VF2	VF2	VF2	VF2	Nauty	Nauty
200	VF2	Nauty	Nauty	VF2	VF2	VF2	VF2	VF2	VF2	Nauty
400	Nauty	Nauty	Nauty	VF2	VF2	VF2	VF2	VF2	VF2	Nauty
600	Nauty	Nauty	Nauty	VF2	VF2	VF2	VF2	VF2	VF2	Nauty
800	Nauty	Nauty	Nauty	VF2	VF2	VF2	VF2	VF2	VF2	VF2
1000	Nauty	Nauty	Nauty	VF2	VF2	VF2	VF2	VF2	VF2	VF2

- 在随机化连接的图中，Nauty算法的表现更好，并且随机化密度越高其效果越好。
- 在正则化程度高的图中，VF2算法表现更佳，尤其是在更大规模的图中。
- 图中一个没表现的信息：在Nauty算法表现最优的情况，VF2算法总是表现次优。

实验结果

带属性值的图-子图同构

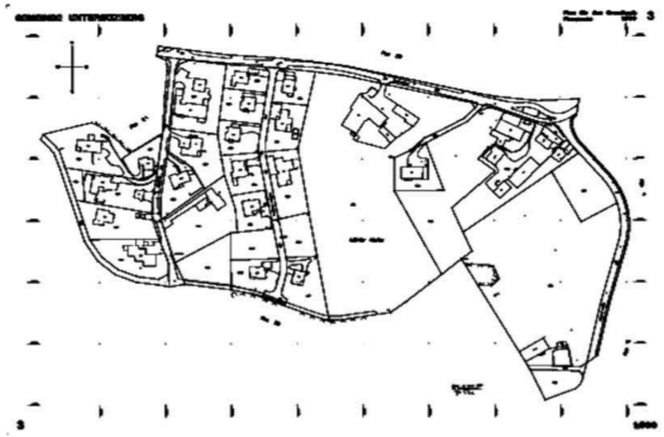
- VF2算法的运行时间和结点数量相对独立

实验结果

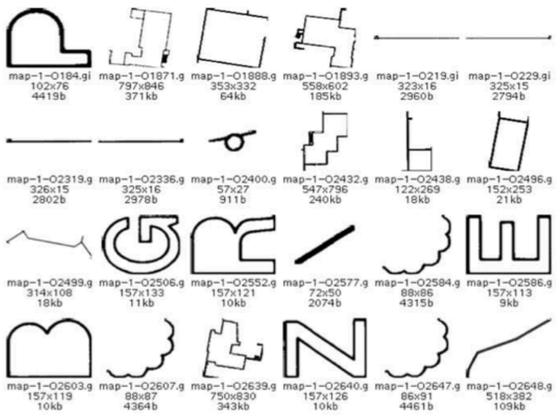
带属性值的图-子图同构

TABLE 3
Some Features of Graphs and Subgraphs Obtained from the Test Images

Image	Type	No. of Graph Nodes	No. of Graph Branches	No. of Subgraphs	No. of Subgraph Nodes
MAP-1	cadastral map	4484	4968	163	2-233
ENGINE-2	mech. drawing	1953	2169	120	2-542



(a)



(b)

- VF2算法的运行时间和结点数量相对独立

实验结果

带属性值的图-子图同构

- VF2算法的运行时间和结点数量相对独立

TABLE 3
Some Features of Graphs and Subgraphs Obtained from the Test Images

Image	Type	No. of Graph Nodes	No. of Graph Branches	No. of Subgraphs	No. of Subgraph Nodes
MAP-1	cadastral map	4484	4968	163	2-233
ENGI[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

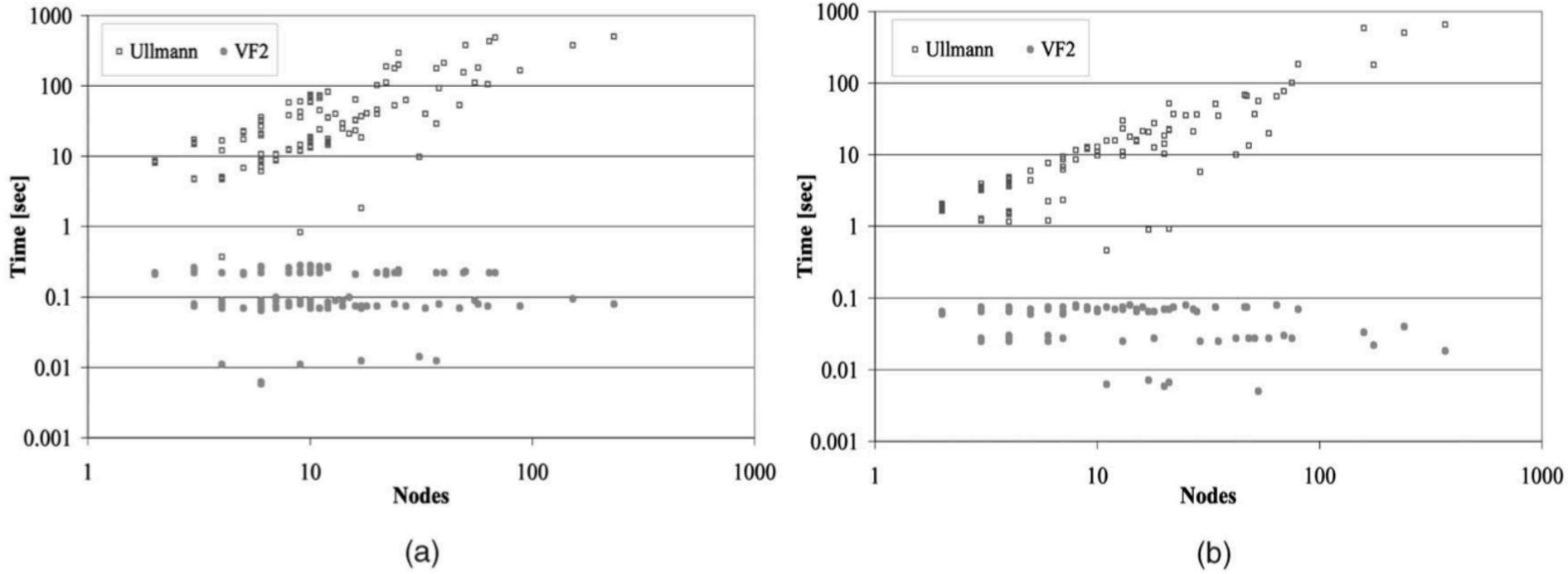


Fig. 3. Matching times for the subgraphs of the MAP-1 image (a) and of the ENGINE-2 image (b).

结论

- VF2算法利用可行性规则进行剪枝，从而减小了计算复杂度。
- VF2算法无需假定待匹配图的结构，对一切输入图都可以进行同构匹配或图-子图同构匹配。
- VF2算法支持带属性关联的图的匹配，并且还可以充分利用属性值来进行剪枝。
- 到VF2算法出现为止，几乎没有算法能兼具以上优点。

谢谢观看！